

4 Decisions

Figure 4.1 credit: modification of work "Fork In The Road", by Ian Sane/Flickr, CC BY 2.0

Chapter Outline

4.1 Boolean values
4.2 If-else statements
4.3 Boolean operations
4.4 Operator precedence
4.5 Chained decisions
4.6 Nested decisions
4.7 Conditional expressions
4.8 Chapter summary

Introduction

The Python interpreter follows a single path of execution when executing a program. What if a programmer wants to define multiple possible paths? Ex: Instead of always taking the left path, a program uses the path width to decide which path to take. If the left path is wider, take the right path. Else, take the left path.

A **branch** is a group of statements that execute based on a condition. The Expressions chapter introduced expressions. This chapter explores how expressions can be used as conditions to make decisions in programs.

4.1 Boolean values

Learning objectives

By the end of this section you should be able to

- Explain a Boolean value.
- Use bool variables to store Boolean values.
- Demonstrate converting integers, floats, and strings to Booleans.
- Demonstrate converting Booleans to integers, floats, and strings.
- Use comparison operators to compare integers, floats, and strings.

bool data type

People often ask binary questions such as yes/no or true/false questions. Ex: Do you like pineapple on pizza? Ex: True or false: I like pineapple on pizza. The response is a Boolean value, meaning the value is either true or false. The **bool** data type, standing for Boolean, represents a binary value of either true or false. `true` and `false` are keywords, and capitalization is required.

CHECKPOINT

Example: Crosswalk sign

Example: Crosswalk sign; ch 4, video 1 (https://www.openstax.org/r/crosswalk-sign)

CONCEPTS IN PRACTICE

Using Boolean variables

Consider the following code:

```
is_fruit = "True"
is_vegetable = 0
is_dessert = False
```

1. What is the data type of `is_fruit`?
   a. Boolean
   b. integer
   c. string
2. What is the data type of `is_vegetable`?
   a. Boolean
   b. integer
   c. string
3. What is the data type of `is_dessert`?
   a. Boolean
   b. integer
   c. string
4. How many values can a Boolean variable represent?
   a. 2
   b. 4
   c. 8
5. Which is a valid value for a Boolean variable?
   a. `true`
   b. `True`
   c. `1`
6. Suppose the following is added to the code above:

   ```
   is_dessert = 0
   ```

```
print(type(is_dessert))
```

What is the output?
a. `<class 'bool'>`
b. `<class 'int'>`
c. `Error`

Type conversion with bool()

Deciding whether a value is true or false is helpful when writing programs/statements based on decisions. Converting data types to Booleans can seem unintuitive at first. Ex: `Is "ice cream" True?` But the conversion is actually simple.

bool() converts a value to a Boolean value, `True` or `False`.

- `True`: any non-zero number, any non-empty string
- `False`: 0, empty string

CHECKPOINT

Converting integers, floats, and strings using bool()

Converting integers, floats, and strings using bool(); ch 4, video 2 (https://www.openstax.org/r/converting-using-bool)

CONCEPTS IN PRACTICE

Converting numeric types and strings to Booleans

7. `bool(0.000)`
   a. `True`
   b. `False`
8. `bool(-1)`
   a. `True`
   b. `False`
9. `bool("")`
   a. `True`
   b. `False`
10. `bool("0")`
    a. `True`
    b. `False`
11. Given input `False`, what is `bool(input())`?
    a. `True`
    b. `False`

CONCEPTS IN PRACTICE

Converting Booleans to numeric types and strings

Given `is_on = True`, what is the value of each expression?

12. `float(is_on)`
- **a.** `0.0`
- **b.** `1.0`

13. `str(is_on)`
- **a.** `"is_on"`
- **b.** `"True"`

14. `int(is_on)`
- **a.** `0`
- **b.** `1`

Comparison operators

Programmers often have to answer questions like "Is the current user the admin?" A programmer may want to compare a string variable, user, to the string, "admin". **Comparison operators** are used to compare values, and the result is either true or false. Ex: `is_admin = (user == "admin")`. `user` is compared with `"admin"` using the == operator, which tests for equality. The Boolean variable, `is_admin`, is assigned with the Boolean result.

The 6 comparison operators:

- equal to: **==**
- not equal to: **!=**
- greater than: **>**
- less than: **<**
- greater than or equal to: **>=**
- less than or equal to: **<=**

CHECKPOINT

Example: Rolling a d20 in a tabletop game

Example: Rolling a d20 in a tabletop game; ch 4, video 3 (https://www.openstax.org/r/rolling-d20)

CONCEPTS IN PRACTICE

Comparing values

For each new variable, what is the value of `compare_result`?

15.
```
x = 14
compare_result = (x <= 13)
```

- **a.** `True`

b. False

16. ```
    w = 0
    compare_result = (w != 0.4)
    ```

    a. True
    b. False

17. ```
    v = 4
    compare_result = (v < 4.0)
    ```

    a. True
    b. False

18. ```
    y = 2
    compare_result = (y > "ab")
    ```

    a. True
    b. False
    c. Error

19. ```
    z = "cilantro"
    compare_result = (z == "coriander")
    ```

    a. True
    b. False

20. ```
    a = "dog"
    compare_result = (a < "cat")
    ```

    a. True
    b. False

= VS ==

A common mistake is using = for comparison instead of ==. Ex: `is_zero = num=0` will always assign `is_zero` and `num` with `0`, regardless of num's original value. The = operator performs assignment and will modify the variable. The == operator performs comparison, does not modify the variable, and produces `True` or `False`.

EXPLORING FURTHER

- Unicode Basic Latin Chart (https://openstax.org/r/100unicodelatin)
    ```

TRY IT

Friday Boolean

"It's Friday, I'm in love" —from "Friday I'm in Love," a song released by the Cure in 1992.

Write a program that reads in the day of the week. Assign the Boolean variable, `in_love`, with the result of whether the day is Friday or not.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-1-boolean-values)

TRY IT

Even numbers

Write a program that reads in an integer and prints whether the integer is even or not. Remember, a number is even if the number is divisible by 2. To test this use `number % 2 == 0`. Ex: If the input is `6`, the output is `"6 is even: True"`.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-1-boolean-values)

4.2 If-else statements

Learning objectives

By the end of this section you should be able to

- Identify which operations are performed when a program with `if` and `if-else` statements is run.
- Identify the components of an `if` and `if-else` statement and the necessary formatting.
- Create an `if-else` statement to perform an operation when a condition is true and another operation otherwise.

if statement

If the weather is rainy, grab an umbrella! People make decisions based on conditions like if the weather is rainy, and programs perform operations based on conditions like a variable's value. Ex: A program adds two numbers. If the result is negative, the program prints an error.

A **condition** is an expression that evaluates to true or false. An **if statement** is a decision-making structure that contains a condition and a body of statements. If the condition is true, the body is executed. If the condition is false, the body is not executed.

The `if` statement's body must be grouped together and have one level of indentation. The PEP 8 style guide recommends four spaces per indentation level. The Python interpreter will produce an error if the body is empty.

CHECKPOINT

Example: Quantity check

Example: Quantity check; ch 4, video 4 (https://www.openstax.org/r/quantity-check)

USING BOOLEAN VARIABLES

A Boolean variable already has a value of `True` or `False` and can be used directly in a condition rather than using the equality operator. Ex: `if is_raining == True:` can be simplified to `if is_raining:`.

CONCEPTS IN PRACTICE

Using if statements

1. Given the following, which part is the condition?

```
if age < 12:
  print("Discount for children available")
```

   a. `age`
   b. `age < 12`
   c. `print("Discount for children available")`

2. Given the following, which lines execute if the condition is true?

```
1 print("Have a great day.")
2 if is_raining:
3   print("Don't forget an umbrella!")
4 print("See you soon.")
```

   a. 1, 2, 3
   b. 1, 2, 4
   c. 1, 2, 3, 4

3. Given the following (same as above), which lines execute if the condition is False?

```
1 print("Have a great day.")
2 if is_raining:
3   print("Don't forget an umbrella!")
4 print("See you soon.")
```

   a. 1, 2, 3
   b. 1, 2, 4
   c. 1, 2, 3, 4

4. Given `num = -10`, what is the final value of `num`?

```
if num < 0:
  num = 25
if num < 100:
```

```
  num = num + 50
```

a. -10
b. 40
c. 75

5. Given input 10, what is the final value of positive_num?

```
positive_num = int(input("Enter a positive number:"))
if positive_num < 0:
  print("Negative input set to 0")
positive_num = 0
```

a. 10
b. 0
c. Error

if-else statement

An `if` statement defines actions to be performed when a condition is true. What if an action needs to be performed only when the condition is false? Ex: If the restaurant is less than a mile away, we'll walk. Else, we'll drive.

An **else statement** is used with an `if` statement and contains a body of statements that is executed when the `if` statement's condition is false. When an `if-else` statement is executed, one and only one of the branches is taken. That is, the body of the `if` or the body of the `else` is executed. Note: The `else` statement is at the same level of indentation as the `if` statement, and the body is indented.

`if-else` statement template:

```
# Statements before

if condition:
  # Body
else:
  # Body

# Statements after
```

CHECKPOINT

Example: Trivia question

Example: Trivia question; ch 4, video 5 (https://www.openstax.org/r/trivia-question)

CONCEPTS IN PRACTICE

Exploring if-else statements

6. Given the following code, the `else` branch is taken for which range of x?

```
if x >= 15:
  # Do something
else:
  # Do something else
```

a. `x >= 15`
b. `x <= 15`
c. `x < 15`

7. Given `x = 40`, what is the final value of y?

```
if x > 30:
  y = x - 10
else:
  y = x + 10
```

a. `30`
b. `40`
c. `50`

8. Given `y = 50`, which is *not* a possible final value of y?

```
if x < 50:
  y = y / 2
else:
  y = y * 2
y = y + 5
```

a. `30`
b. `55`
c. `105`

TRY IT

Improved division

The following program divides two integers. Division by 0 produces an error. Modify the program to read in a new denominator (with no prompt) if the denominator is 0.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-2-if-else-statements)

TRY IT

Converting temperature units

The following program reads in a temperature as a float and the unit as a string: "f" for Fahrenheit or "c" for Celsius.

Calculate `new_temp`, the result of converting temp from Fahrenheit to Celsius or Celsius to Fahrenheit based on unit. Calculate `new_unit`: "c" if unit is "f" and "f" if unit is "c".

Conversion formulas:

- Degrees Celsius = (degrees Fahrenheit - 32) * 5/9
- Degrees Fahrenheit = (degrees Celsius * 5/9) + 32

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-2-if-else-statements)

4.3 Boolean operations

Learning objectives

By the end of this section you should be able to

- Explain the purpose of logical operators.
- Describe the truth tables for `and`, `or`, and `not`.
- Create expressions with logical operators.
- Interpret `if-else` statements with conditions using logical operators.

Logical operator: and

Decisions are often based on multiple conditions. Ex: A program printing if a business is open may check that `hour >= 9` and `hour < 17`. A **logical operator** takes condition operand(s) and produces `True` or `False`.

Python has three logical operators: *and*, *or*, and *not*. The **and** operator takes two condition operands and returns `True` if both conditions are true.

p	q	p and q
True	True	True
True	False	False
False	True	False
False	False	False

Table 4.1 Truth table: p and q.

CHECKPOINT

Example: Museum entry

Example: Museum entry; ch 4, video 6 (https://www.openstax.org/r/museum-entry)

CONCEPTS IN PRACTICE

Using the and operator

1. Consider the example above. Jaden tries to enter when the capacity is 2500 and there are 2 hours before close. Can Jaden enter?
   a. yes
   b. no
2. Consider the example above. Darcy tries to enter when the capacity is 3000. For what values of `hrs_to_close` will Darcy to be able to enter?
   a. `hrs_to close > 1.0`
   b. no such value
3. Given `is_admin = False` and `is_online = True`, what is the value of `is_admin` and `is_online`?
   a. `True`
   b. `False`
4. Given `x = 8` and `y = 21`, what is the final value of z?

```
if (x < 10) and (y > 20):
  z = 5
else:
  z = 0
```

   a. `0`
   b. `5`

Logical operator: or

Sometimes a decision only requires one condition to be true. Ex: If a student is in the band or choir, they will perform in the spring concert. The **or** operator takes two condition operands and returns `True` if either condition is `true`.

p	q	p or q
True	True	True
True	False	True
False	True	True
False	False	False

Table 4.2 Truth table: p or q.

CHECKPOINT

Example: Streaming prompt

Example: Streaming prompt; ch 4, video 7 (https://www.openstax.org/r/streaming-prompt)

CONCEPTS IN PRACTICE

Using the or operator

5. Given `days = 21` and `is_damaged` is `False`, is the refund processed?

```
if (days < 30) or is_damaged:
  # Process refund
```

   a. yes
   b. no

6. For what values of age is there no discount?

```
if (age < 12) or (age > 65):
  # Apply student/senior discount
```

   a. `age >= 12`
   b. `age <= 65`
   c. `(age >= 12)` and `(age <= 65)`

7. Given `a = 9` and `b = 10`, does the test pass?

```
if (a%2 == 0 and b%2 == 1) or (a%2 == 1 and b%2 == 0):
  # Test passed
else:
  # Test failed
```

   a. yes
   b. no

Logical operator: not

If the computer is not on, press the power button. The **not** operator takes one condition operand and returns `True` when the operand is false and returns `False` when the operand is true.

not is a useful operator that can make a condition more readable and can be used to toggle a Boolean's value. Ex: `is_on = not is_on`.

p	not p
True	False
False	True

Table 4.3 Truth table: not p.

CHECKPOINT

Example: Diving warning

Example: Diving warning; ch 4, video 8 (https://www.openstax.org/r/diving-warning)

CONCEPTS IN PRACTICE

Using the not operator

8. Given `x = 13`, what is the value of `not(x < 10)`?
   a. `True`
   b. `False`

9. Given `x = 18`, is x in the correct range?

```
if not(x > 15 and x < 20):
  # x in correct range
```

   a. yes
   b. no

10. Given `is_turn = False` and `timer = 65`, what is the final value of `is_turn`?

```
if timer > 60:
  is_turn = not is_turn
```

   a. `True`
   b. `False`

TRY IT

Speed limits

Write a program that reads in a car's speed as an integer and checks if the car's speed is within the freeway limits. A car's speed must be at least 45 mph but no greater than 70 mph on the freeway.

If the speed is within the limits, print "Good driving". Else, print "Follow the speed limits".

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-3-boolean-operations)

4.4 Operator precedence

Learning objectives

By the end of this section you should be able to

- Describe how precedence impacts order of operations.
- Describe how associativity impacts order of operations.
- Explain the purpose of using parentheses in expressions with multiple operators.

Precedence

When an expression has multiple operators, which operator is evaluated first? Precedence rules provide the priority level of operators. Operators with the highest precedence execute first. Ex: `1 + 2 * 3` is `7` because multiplication takes precedence over addition. However, `(1 + 2) * 3` is `9` because parentheses take precedence over multiplication.

Operator	Meaning
()	Parentheses
**	Exponentiation (right associative)
*, /, //, %	Multiplication, division, floor division, modulo
+, -	Addition, subtraction
<, <=, >, >=, ==, !=	Comparison operators
not	Logical `not` operator
and	Logical `and` operator
or	Logical `or` operator

Table 4.4 Operator precedence from highest to lowest.

CHECKPOINT

Operator precedence

Operator precedence; ch 4, video 9 (https://www.openstax.org/r/operator-precedence)

CONCEPTS IN PRACTICE

Precedence rules

Which part of each expression is evaluated first?

1. `x ** 2 + 6 / 3`
   a. `6 / 3`
   b. `x ** 2`
   c. `2 + 6`
2. `not 3 * 5 > 10`
   a. `3 * 5`
   b. `not 3`
   c. `5 > 10`
3. `z == 5 and x / 8 < 100`
   a. `5 and x`
   b. `x / 8`
   c. `8 < 100`

Associativity

What if operators beside each other have the same level of precedence? Associativity determines the order of operations when precedence is the same. Ex: `8 / 4 * 3` is evaluated as `(8/4) * 3` rather than `8 / (4*3)` because multiplication and division are left associative. Most operators are left associative and are evaluated from left to right. Exponentiation is the main exception (noted above) and is right associative: that is, evaluated from right to left. Ex: `2 ** 3 ** 4` is evaluated as `2 ** (3**4)`.

When comparison operators are chained, the expression is converted into the equivalent combination of comparisons and evaluated from left to right. Ex. `10 < x <= 20` is evaluated as `10 < x` and `x <= 20`.

CHECKPOINT

Operation precedence

Operation precedence; ch 4, video 10 (https://www.openstax.org/r/operation-precedence)

CONCEPTS IN PRACTICE

Associativity

How is each expression evaluated?

4. `10 + 3 * 2 / 4`
   a. `10 + (3 * (2 / 4))`
   b. `10 + ((3 * 2) / 4)`
   c. `(10 + 3) * (2 / 4)`
5. `2 * 2 ** 2 ** 3`
   a. `2 * ((2 ** 2) ** 3)`
   b. `2 * (2 ** (2 ** 3))`
   c. `((2*2) ** 2) ** 3`
6. `100 < x > 150`
   a. `100 < x and x < 150`

b. `100 < x or x > 150`
c. `100 < x and x > 150`

Enforcing order and clarity with parentheses

Operator precedence rules can be hard to remember. Parentheses not only assert a different order of operations but also reduce confusion.

CHECKPOINT

Using parentheses

Using parenthesis; ch 4, video 11 (https://www.openstax.org/r/using-parentheses)

CONCEPTS IN PRACTICE

Using parentheses

7. Consider the example above. Why was the evaluation order different from what the programmer wanted?
a. Equality has precedence over `and`.
b. All operators are evaluated right to left.
c. Order is random when parentheses aren't used.

8. Given `x = 8` and `y = 9`, what is the result of the following?
`x + 3 * y - 5`
a. `30`
b. `44`
c. `94`

9. Given `x = 8` and `y = 9`, what is the result of the following?
`(x+3) * (y-5)`
a. `30`
b. `44`
c. `94`

PEP 8 RECOMMENDATIONS: SPACING AROUND OPERATORS

The PEP 8 style guide recommends consistent spacing around operators to avoid extraneous and confusing whitespace.

- Avoid multiple spaces and an unequal amount of whitespace around operators with two operands.
  Avoid: `x= y * 44`
  Better: `x = y * 44`
- Avoid spaces immediately inside parentheses.
  Avoid: `x = ( 4 * y )`
  Better: `x = (4 * y)`
- Surround the following operators with one space: assignment, augment assignment, comparison,

Boolean.
Avoid: x= y<44
Better: x = y < 44

- Consider adding whitespace around operators with lower priority.
  Avoid: x = 5 * z+20
  Better: x = 5*z + 20

4.5 Chained decisions

Learning objectives

By the end of this section you should be able to

- Identify the branches taken in an `if-elif` and `if-elif-else` statement.
- Create a chained decision statement to evaluate multiple conditions.

elif

Sometimes, a complicated decision is based on more than a single condition. Ex: A travel planning site reviews the layovers on an itinerary. If a layover is greater than 24 hours, the site should suggest accommodations. *Else if* the layover is less than one hour, the site should alert for a possible missed connection.

Two separate `if` statements do not guarantee that only one branch is taken and might result in both branches being taken. Ex: The program below attempts to add a curve based on the input test score. If the input is `60`, both `if` statements are incorrectly executed, and the resulting score is 75.

```
score = int(input())
if score < 70:
  score += 10
# Wrong:
if 70 <= score < 85:
  score += 5
```

Chaining decision statements with `elif` allows the programmer to check for multiple conditions. An **elif** (short for else if) statement checks a condition when the prior decision statement's condition is false. An `elif` statement is part of a chain and must follow an `if` (or `elif`) statement.

`if-elif` statement template:

```
# Statements before

if condition:
  # Body
elif condition:
  # Body

# Statements after
```

CHECKPOINT

Example: Livestream features

Example: Livestream features; ch 4, video 12 (https://www.openstax.org/r/livestream)

CONCEPTS IN PRACTICE

Using elif

1. Fill in the blank to execute Body 2 when condition_1 is false and condition_2 is true.

```
if condition_1:
  # Body 1
__ condition_2:
  # Body 2
```

   a. if
   b. elif
   c. else

2. Given x = 42 and y = 0, what is the final value of y?

```
if x > 44:
  y += 2
elif x < 50:
  y += 5
```

   a. 2
   b. 5
   c. 7

3. Which conditions complete the code such that if x is less than 0, Body 1 executes, else if x equals 0, Body 2 executes.

```
if _________:
  # Body 1
elif _________:
  # Body 2
```

   a. x < 0
      x == 0
   b. x == 0
      x < 0
   c. x <= 0
      [no condition]

4. Which of the following is a valid chained decision statement?
   a.
```
if condition_1:
    # Body 1
elif condition_2:
```

```
    # Body 2
```

b.
```
if condition_1:
    # Body 1
    elif condition_2:
      # Body 2
```

c.
```
elif condition_1:
    # Body 1
  if condition_2:
    # Body 2
```

5. Given `attendees = 350`, what is the final value of `rooms`?

```
rooms = 1
if attendees >= 100:
  rooms += 3
if attendees <= 200:
  rooms += 7
elif attendees <= 400:
  rooms += 14
```

a. 4
b. 15
c. 18

if-elif-else statements

Elifs can be chained with an `if-else` statement to create a more complex decision statement. Ex: A program shows possible chess moves depending on the piece type. If the piece is a pawn, show moving forward one (or two) places. Else if the piece is a bishop, show diagonal moves. Else if . . . (finish for the rest of the pieces).

CHECKPOINT

Example: Possible chess moves

Example: Possible chess moves; ch 4, video 13 (https://www.openstax.org/r/chess-moves)

CONCEPTS IN PRACTICE

Using elif within if-elif-else statements

6. Given `hour = 12`, what is printed?

```
if hour < 8:
  print("Too early")
elif hour < 12:
  print("Good morning")
elif hour < 13:
  print("Lunchtime")
```

```
elif hour < 17:
  print("Good afternoon")
else:
  print("Too late")
```

a. `Good morning`
b. `Lunchtime`
c. `Good afternoon`
d. `Too late`

7. Where can an `elif` statement be added?

```
_1_
if condition:
  # Body
  _2_
elif condition:
  # Body
_3_
else:
  # Body
_4_
```

a. 1
b. 2
c. 3
d. 4

8. Given `x = -1` and `y = -2`, what is the final value of y?

```
if x < 0 and y < 0:
  y = 10
elif x < 0 and y > 0:
  y = 20
else:
  y = 30
```

a. `10`
b. `20`
c. `30`

9. How could the following statements be rewritten as a chained statement?

```
if price < 9.99:
  order = 50
if 9.99 <= price < 19.99:
  order = 30
if price >= 19.99:
  order = 10
```

a. `if price < 9.99:`

```
      order = 50
    else:
      order = 30
    order = 10
b. if price < 9.99:
     order = 50
   elif price < 19.99:
     order = 30
   elif price == 19.99:
     order = 10
c. if price < 9.99:
     order = 50
   elif price < 19.99:
     order = 30
   else:
     order = 10
```

TRY IT

Crochet hook size conversion

Write a program that reads in a crochet hook's US size and computes the metric diameter in millimeters. (A subset of sizes is used.) If the input does not match B-G, the diameter should be assigned with -1.0. Ex: If the input is D, the output is `"3.25 mm"`.

Size conversions for US size: mm

- B : 2.25
- C : 2.75
- D : 3.25
- E : 3.5
- F : 3.75
- G : 4.0

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-5-chained-decisions)

TRY IT

Color wavelengths

Write a program that reads in an integer representing a visible light wavelength in nanometers. Print the corresponding color using the following inclusive ranges:

- Violet: 380–449
- Blue: 450–484
- Cyan: 485–499
- Green: 500–564
- Yellow: 565–589

- Orange: 590–624
- Red: 625–750

Assume the input is within the visible light spectrum, 380-750 inclusive.

Given input:

```
550
```

The output is:

```
Green
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-5-chained-decisions)

4.6 Nested decisions

Learning objectives

By the end of this section you should be able to

- Describe the execution paths of programs with nested `if-else` statements.
- Implement a program with nested `if-else` statements.

Nested decision statements

Suppose a programmer is writing a program that reads in a game ID and player count and prints whether the user has the right number of players for the game.

The programmer may start with:

```
if game == 1 and players < 2:
  print("Not enough players")
if game == 1 and players > 4:
  print("Too many players")
if game == 1 and (2 <= players <= 4):
  print("Ready to start")
if game == 2 and players < 3:
  print("Not enough players")
if game == 2 and players > 6:
  print("Too many players")
if game == 2 and (3 <= players <= 6):
  print("Ready to start")
```

The programmer realizes the code is redundant. What if the programmer could decide the game ID first and then make a decision about players? Nesting allows a decision statement to be inside another decision statement, and is indicated by an indentation level.

An improved program:

```
if game == 1:
  if players < 2:
    print("Not enough players")
  elif players > 4:
    print("Too many players")
  else:
    print("Ready to start")
if game == 2:
  if players < 3:
    print("Not enough players")
  elif players > 6:
    print("Too many players")
  else:
    print("Ready to start")
# Test game IDs 3-end
```

CHECKPOINT

Example: Poisonous plant identification

Example: Poisonous plant identification; ch 4, video 14 (https://www.openstax.org/r/plant-identification)

CONCEPTS IN PRACTICE

Using nested if-else statements

1. Consider the example above. Given `leaf_count = 9` and `leaf_shape = "teardrop"`, what is the output?
   a. `Might be poison ivy`
   b. `Might be poison oak`
   c. `Might be poison sumac`

2. Given `num_dancers = 49`, what is printed?

```
if num_dancers < 0:
  print("Error: num_dancers is negative")
else:
  if num_dancers % 2 == 1:
    print("Error: num_dancers is odd")
  print(num_dancers, "dancers")
```

   a. `Error: num_dancers is odd`
   b. `49 dancers`
   c. `Error: num_dancers is odd`
      `49 dancers`

3. Given `x = 256`, `y = 513`, and `max = 512`, which of the following will execute?

```
if x == y:
```

```
    # Body 1
  elif x < y:
    # Body 2
    if y >= max:
      # Body 3
    else:
      # Body 4
  else:
    # Body 5
```

a. Body 2
b. Body 2, Body 3
c. Body 2, Body 5

4. Given x =118, y = 300, and max = 512, which of the following will execute?

```
if x == y:
  # Body 1
elif x < y:
  # Body 2
  if y >= max:
    # Body 3
else:
  # Body 4
else:
  # Body 5
```

a. Body 2
b. Body 3
c. Error

TRY IT

Meal orders

Write a program that reads in a string, "lunch" or "dinner", representing the menu choice, and an integer, 1, 2, or 3, representing the user's meal choice. The program then prints the user's meal choice.

Lunch Meal Options

- 1: Caesar salad
- 2: Spicy chicken wrap
- 3: Butternut squash soup

Dinner Meal Options

- 1: Baked salmon
- 2: Turkey burger
- 3: Mushroom risotto

Ex: If the input is:
`lunch`
`3`

The output is:
`Your order: Butternut squash soup`

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-6-nested-decisions)

4.7 Conditional expressions

Learning objectives

By the end of this section you should be able to

- Identify the components of a conditional expression.
- Create a conditional expression.

Conditional expressions

A **conditional expression** (also known as a "ternary operator") is a simplified, single-line version of an `if-else` statement.

Conditional expression template:

```
expression_if_true if condition else expression_if_false
```

A conditional expression is evaluated by first checking the condition. If `condition` is true, `expression_if_true` is evaluated, and the result is the resulting value of the conditional expression. Else, `expression_if_false` is evaluated, and the result is the resulting value of the conditional expression.

A variable can be assigned with a conditional expression. Ex: Finding the max of two numbers can be calculated with `max_num = y if x < y else x`

Note: Conditional expressions have the lowest precedence of all Python operations.

CHECKPOINT

Example: Version check

Example: Version check; ch 4, video 15 (https://www.openstax.org/r/version-check)

CONCEPTS IN PRACTICE

Using conditional expressions

1. What is the conditional expression version of the following `if-else` statement?

```
if x%2 == 0:
  response = 'even'
```

```
else:
  response = 'odd'
```

a. `response = if x%2 == 0 "even" else "odd"`
b. `response = "odd" if x%2 == 0 else "even"`
c. `response = "even" if x%2 == 0 else "odd"`

2. Given `x = 100` and `offset = 10`, what is the value of `result`?

```
result = x + offset if x < 100 else x - offset
```

a. `90`
b. `100`
c. `110`

3. Which part of the conditional expression is incorrect?

```
min_num = x if x < y else min_num = y
```

a. `min_num = x`
b. `x < y`
c. `min_num = y`

4. Which of the following is an improved version of the following `if-else` statement?

```
if x < 50:
  result = True
else:
  result = False
```

a. `result = True if x < 50 else False`
b. `result = x < 50`

5. What are the possible values of `total`?

```
total = fee + 10 if hours > 12 else 2
```

a. `10`, `2`
b. `fee + 10`, `2`
c. `fee + 10`, `fee + 2`

TRY IT

Ping values

Write a program that reads in an integer, `ping`, and prints `ping_report`, a string indicating whether the ping is low to average or too high. `ping` values under 150 have a `ping_report` of `"low to average"`. `ping` values of 150 and higher have a `ping_report` of `"too high"`. Use a conditional expression to assign `ping_report`.

Ex: If the input is `30`, the output is `"Ping is low to average"`.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/4-7-conditional-expressions)

4.8 Chapter summary

Highlights from this chapter include:

- Booleans represent a value of `True` or `False`.
- Comparison operators compare values and produce `True` or `False`.
- Logical operators take condition operand(s) and produce `True` or `False`.
- Operators are evaluated in order according to precedence and associativity.
- Conditions are expressions that evaluate to `True` or `False`.
- Decision statements allow different paths of execution (branches) through code based on conditions.
- Decision statements can be nested inside other decision statements.
- Conditional expressions are single-line versions of `if-else` statements.

At this point, you should be able to write programs that evaluate conditions and execute code statements accordingly with the correct order of operations. The programming practice below ties together most topics presented in the chapter.

Function	Description
`bool(x)`	Converts x to a Boolean value, either `True` or `False`.
Operator	**Description**
`x == y` (Equality)	Compares the values of x and y and returns `True` if the values are equal and `False` otherwise. Ex: `10 == 10` is `True`.
`x != y` (Inequality)	Compares the values of x and y and returns `True` if the values are inequal and `False` otherwise. Ex: `7 != 4` is `True`.
`x > y` (Greater than)	Compares the values of x and y and returns `True` if the x is greater than y and `False` otherwise. Ex: `9 > 3` is `True`.
`x < y` (Less than)	Compares the values of x and y and returns `True` if the x is less than y and `False` otherwise. Ex: `9 < 8` is `False`.
`x >= y` (Greater than or equal)	Compares the values of x and y and returns `True` if the x is greater than or equal to y and `False` otherwise. Ex: `2 >= 2` is `True`.

Table 4.5 Chapter 4 reference.

Function	Description
`x <= y` (Less than or equal)	Compares the values of x and y and returns `True` if the x is less than or equal to y and `False` otherwise. Ex: `8 <= 7` is `False`.
`x and y` (Logical)	Evaluates the Boolean values of x and y and returns `True` if both are true. Ex: `True and False` is `False`.
`x or y` (Logical)	Evaluates the Boolean values of x and y and returns `True` if either is true. Ex: `True or False` is `True`.
`not x` (Logical)	Evaluates the Boolean value of x and returns `True` if the value is false and `False` if the value is true. Ex: `not True` is `False`.
Decision statement	**Description**

`if` statement	`# Statements before`
	
	`if condition:`
	`  # Body`
	
	`# Statements after`

`else` statement	`# Statements before`
	
	`if condition:`
	`  # Body`
	`else:`
	`  # Body`
	
	`# Statements after`

Table 4.5 Chapter 4 reference.

Function	Description

`elif` statement	`# Statements before`
	
	`if condition:`
	`  # Body`
	`elif condition:`
	`  # Body`
	`else:`
	`  # Body`
	
	`# Statements after`

Nested `if` statement	`# Statements before`
	
	`if condition:`
	`  if condition:`
	`  # Body`
	`  else:`
	`  # Body`
	`else:`
	`  if condition:`
	`  # Body`
	`  else:`
	`  # Body`
	
	`# Statements after`

Conditional expression	`expression_if_true if condition else expression_if_false`

Table 4.5 Chapter 4 reference.